Machine Learning

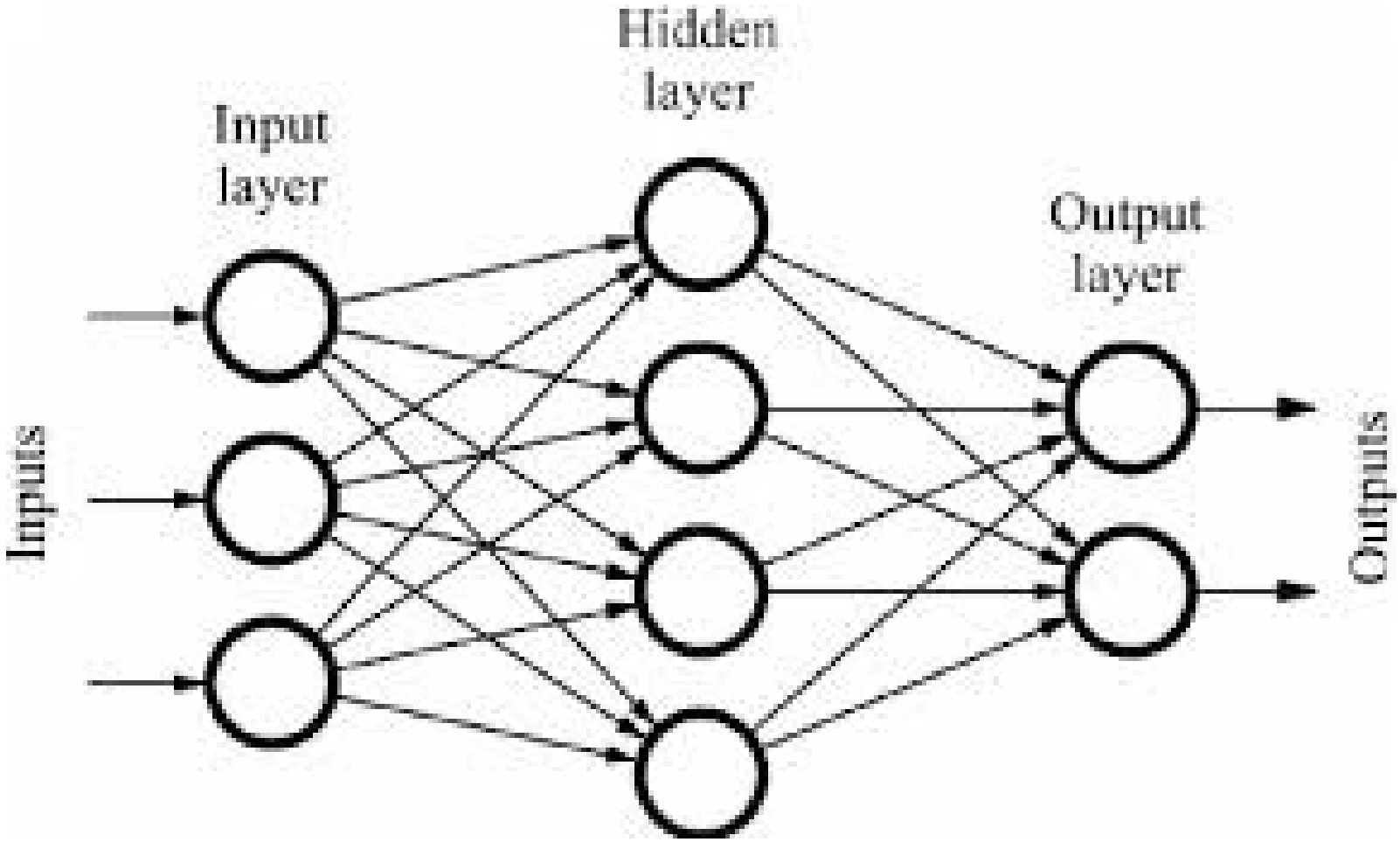
Dr. Irfan Yousuf
Department of Computer Science (New Campus)
UET, Lahore

(Week 12; December 01 - 05, 2025)

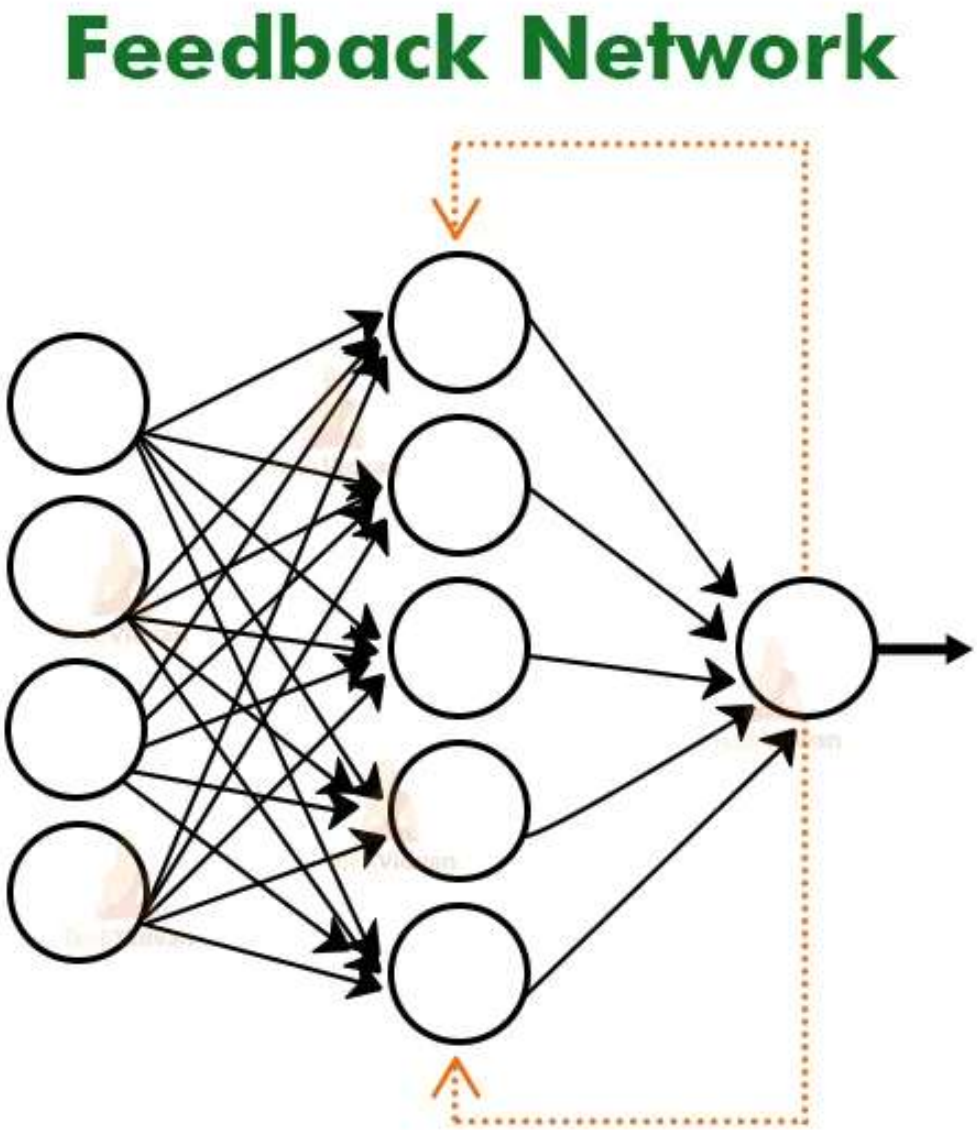
Outline

- Recurrent Neural Networks

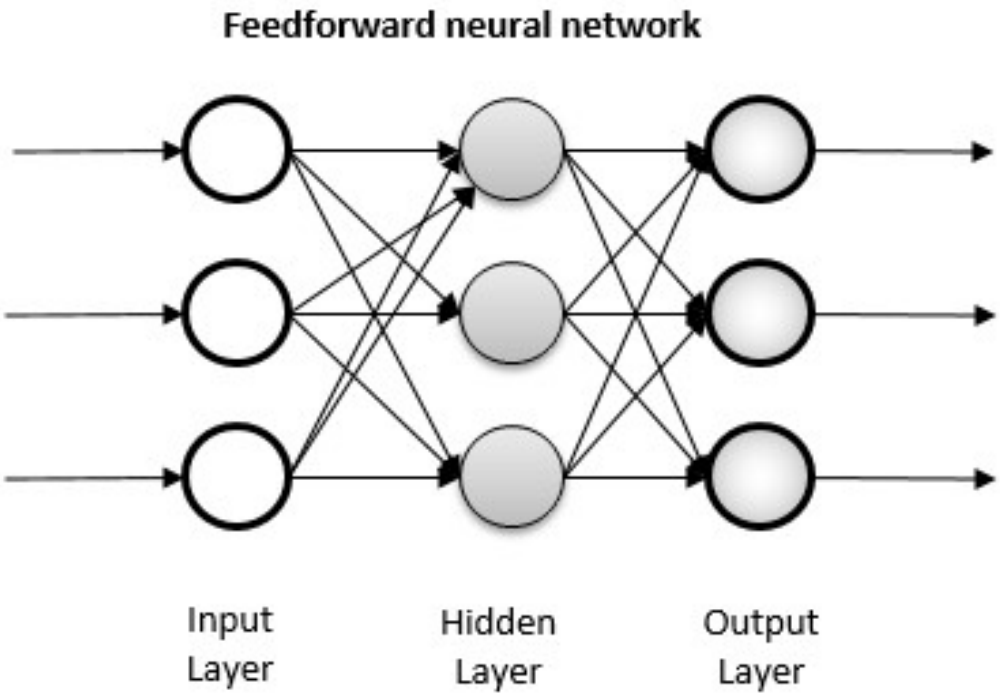
Feedforward Neural Networks

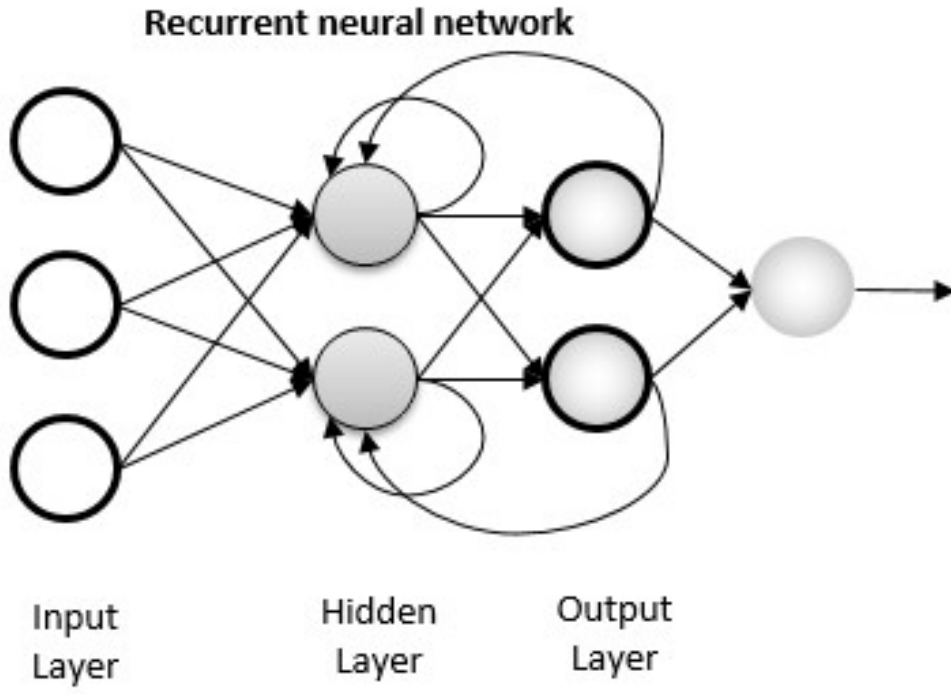


Feedback Neural Networks



Recurrent Neural Networks





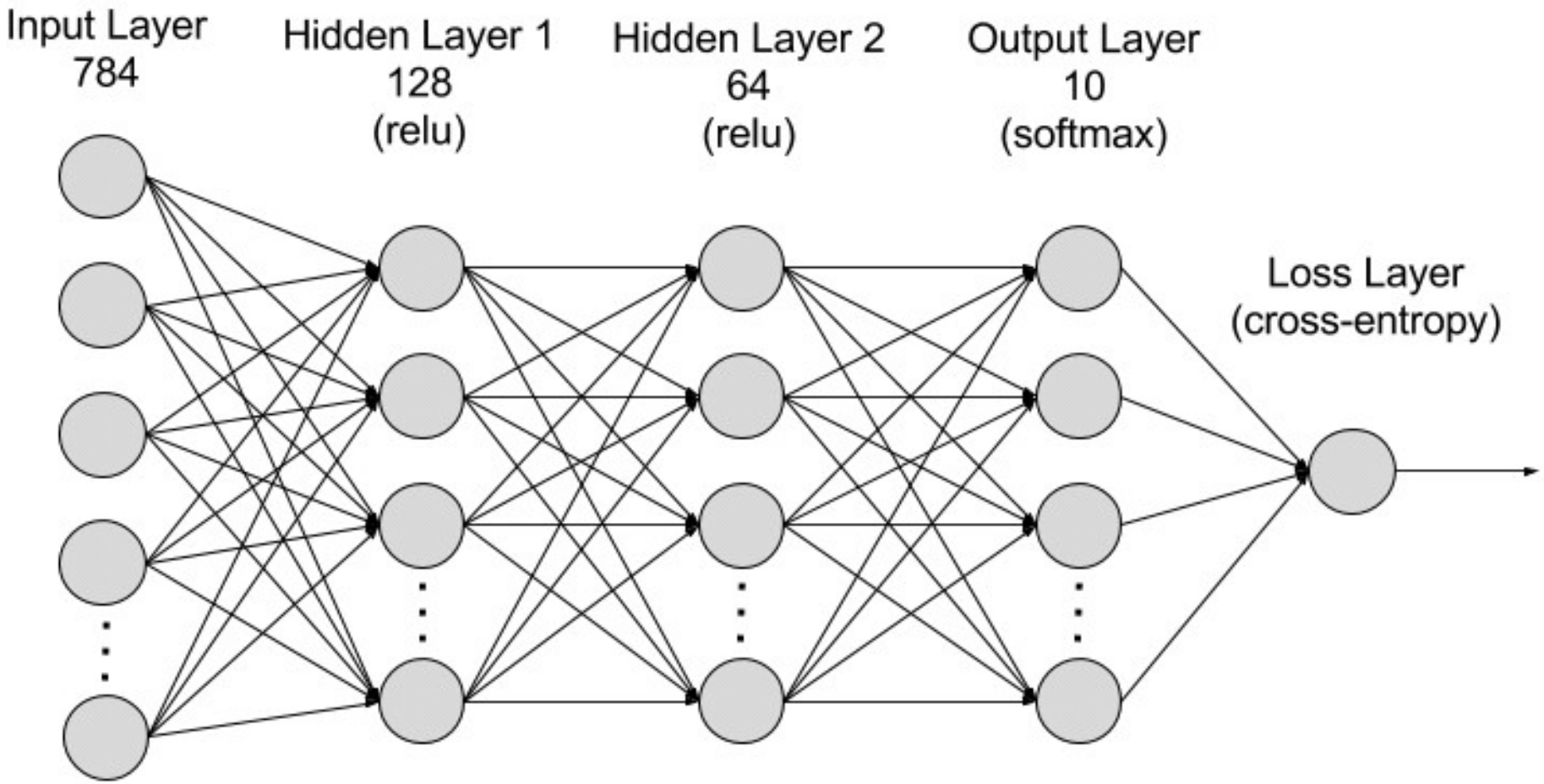
Recurrent Neural Networks

- A recurrent neural network (RNN) is a deep learning rchitecture that is trained to process and convert a **sequential data** input into a specific sequential data output.

- Sequential data is data—such as words, sentences, or time-series data—where sequential components interrelate based on complex semantics and syntax rules.

- RNNs are largely being replaced by transformer-based artificial intelligence (AI) and large language models (LLM), which are much more efficient in sequential data processing.

Recurrent Neural Networks



Recurrent Neural Networks

- RNNs work by passing the sequential data that they receive to the hidden layers one step at a time.
- However, they also have a **self-looping or recurrent workflow**: the hidden layer **can remember** and use previous inputs for future predictions in a **short-term memory component.**
- It uses the current input and the stored memory to predict the next sequence.

Key Characteristics of Recurrent Neural Networks

- **Feedback Loops:** Unlike traditional feedforward neural networks, where information flows in only one direction, RNNs have loops that allow the output of a neuron or layer to be used as input to the same or a previous layer at a subsequent time step.

- **Sequential Data Processing:** This feedback mechanism makes RNNs particularly well-suited for processing sequential data, such as time series, natural language, and speech, where the context and order of data points are crucial for prediction or classification tasks.

Key Characteristics of Recurrent Neural Networks

- **Internal Memory:** The feedback loop allows the network to maintain an internal state, often referred to as a "hidden state" or "memory," which captures information from past inputs to influence the current output.

- **Variants:** Common and more advanced RNN architectures include Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) networks, which were developed to address challenges like the vanishing gradient problem in simple RNNs.

Recurrent Neural Networks

- The building block of RNNs is the **recurrent unit.**
- This unit maintains a hidden state, essentially a form of memory, which is updated at each time step based on the current input and the previous hidden state.
- This feedback loop allows the network to learn from past inputs, and incorporate that knowledge into its current processing.

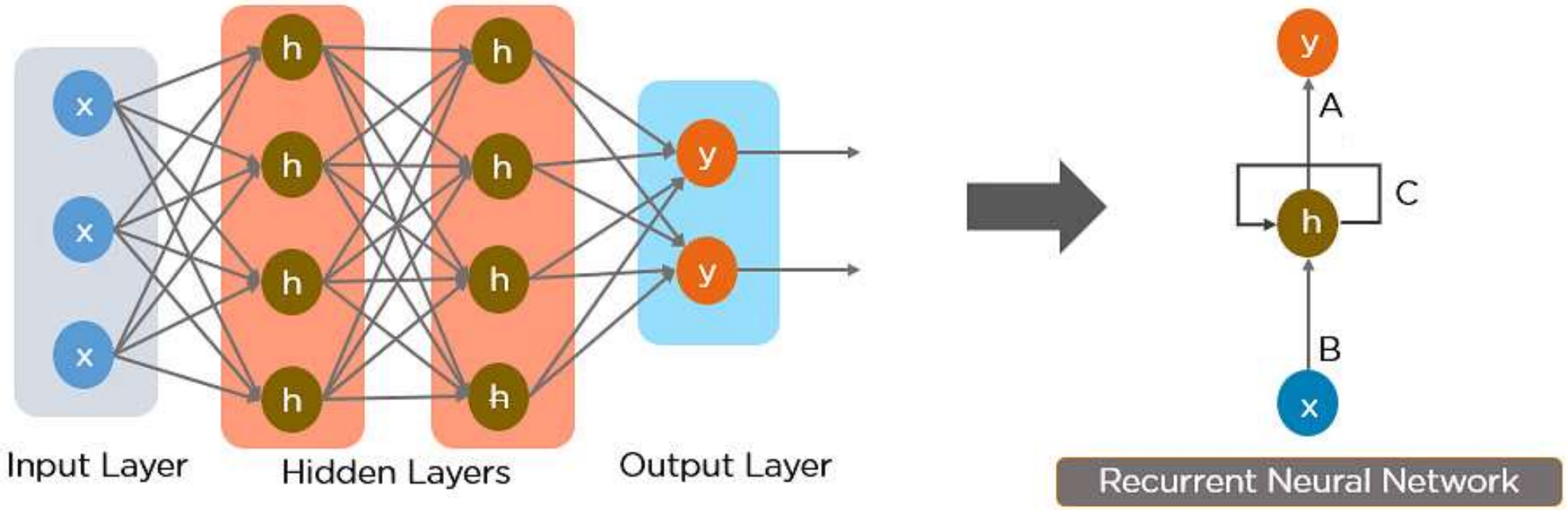
Recurrent Neural Networks

- **For example,** consider the sequence: Apple is red.

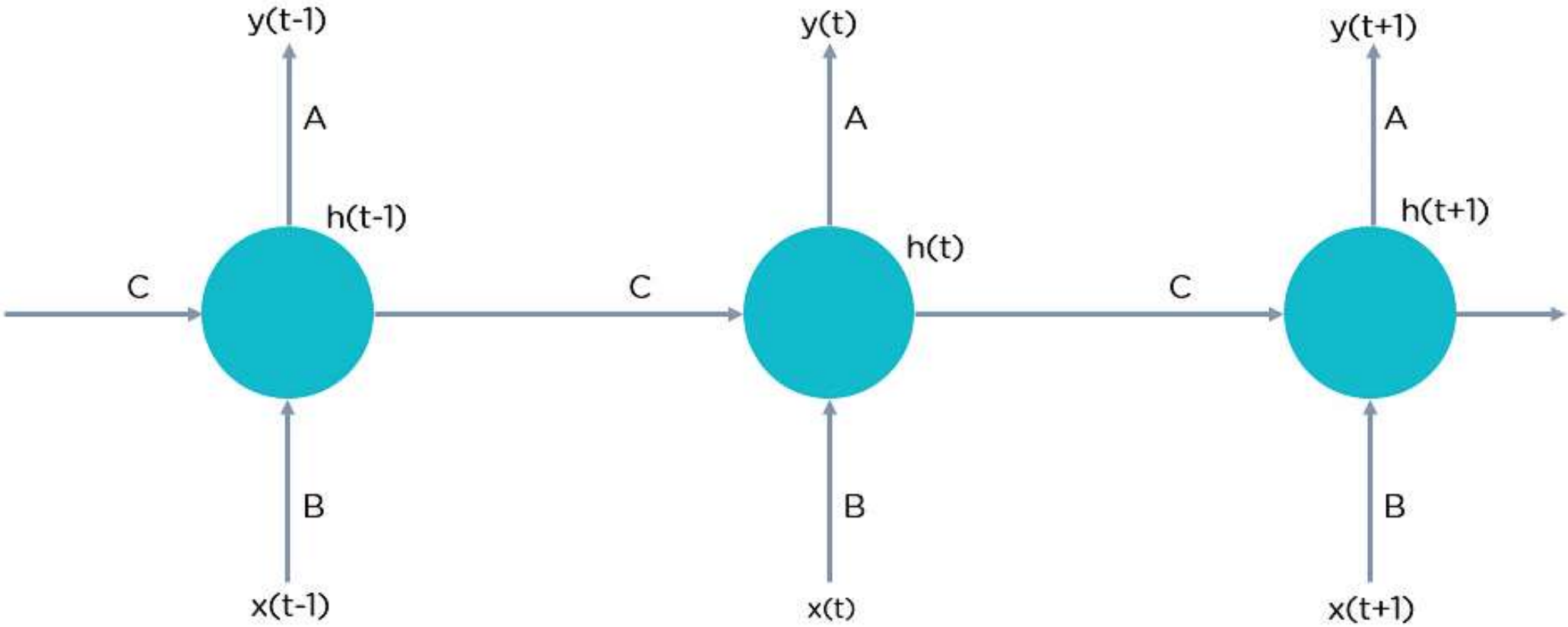
- You want the RNN to predict red when it receives the input sequence Apple is.
- When the hidden layer processes the word Apple, it stores a copy in its memory.
- Next, when it sees the word is, it recalls Apple from its memory and understands the full sequence: Apple is for context.
- It can then predict red for improved accuracy. This makes RNNs useful in speech recognition, machine translation, and other language modeling tasks.

Recurrent Neural Networks

- Recurrent neural networks add the immediate past to the present.
- Therefore, an RNN has **two inputs**: the present and the recent past.



Recurrent Neural Networks

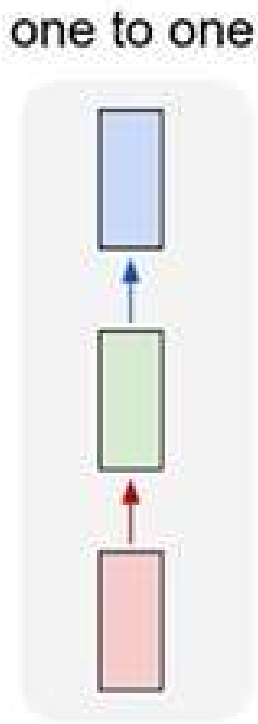


$h(t) = f_c(h(t-1), x(t))$

h(t) = new state
f_c = function with parameter c
h(t-1) = old state
x(t) = input vector at time step t

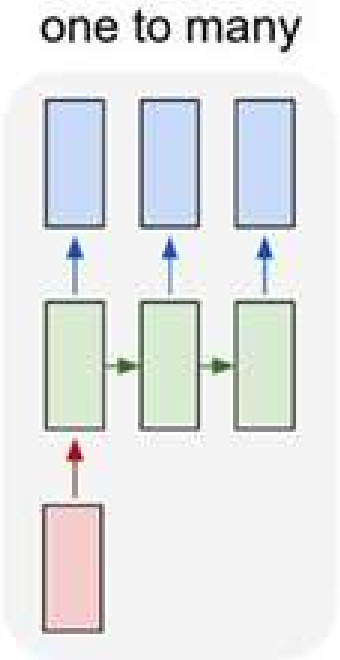
Configurations of Recurrent Neural Networks

- **One-to-one** RNN models have one input and one output. Also called a vanilla neural network, one-to-one architecture is used in traditional neural networks and for general machine learning tasks like image classification.



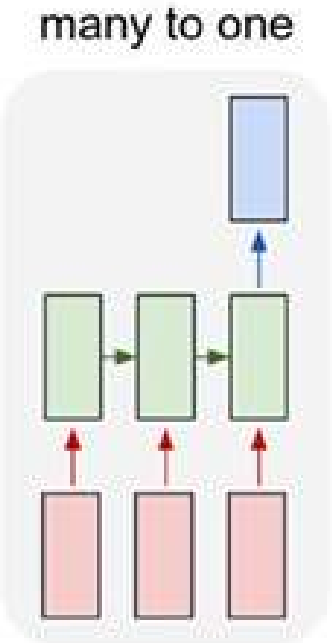
Configurations of Recurrent Neural Networks

- **One-to-many** RNNs have one input and several outputs. This enables image captioning or music generation capabilities, as it uses a single input (like a keyword) to generate multiple outputs (like a sentence).



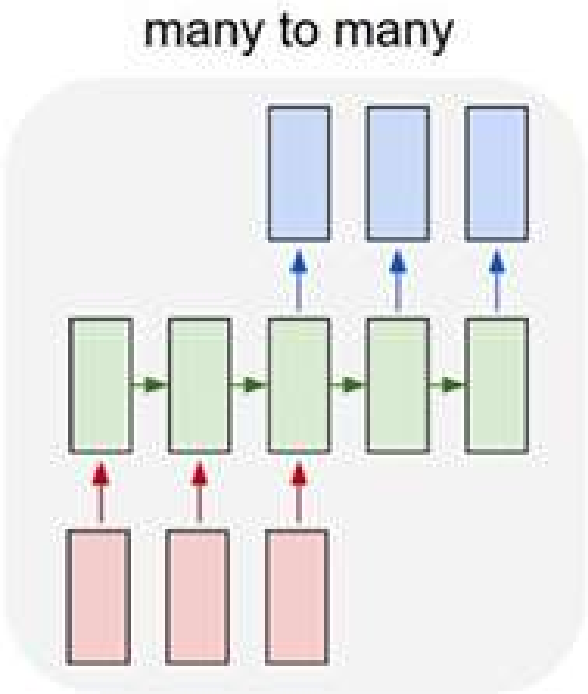
Configurations of Recurrent Neural Networks

- A **many-to-one** model has several inputs and one output. Tasks like sentiment analysis or text classification often use many-to-one architectures. For example, a sequence of inputs (like a sentence) can be classified into one category (like if the sentence is considered a positive/negative sentiment).

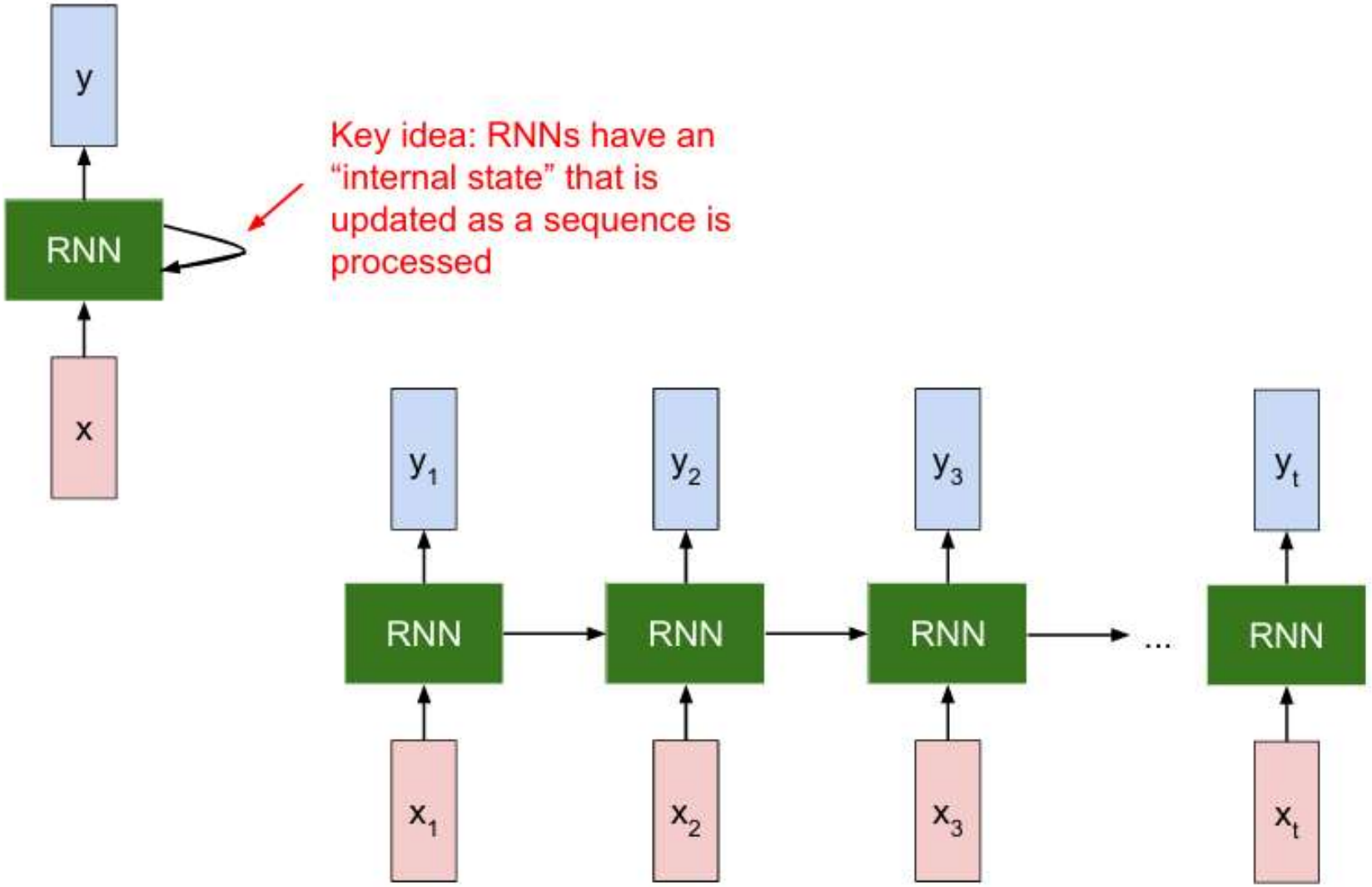


Configurations of Recurrent Neural Networks

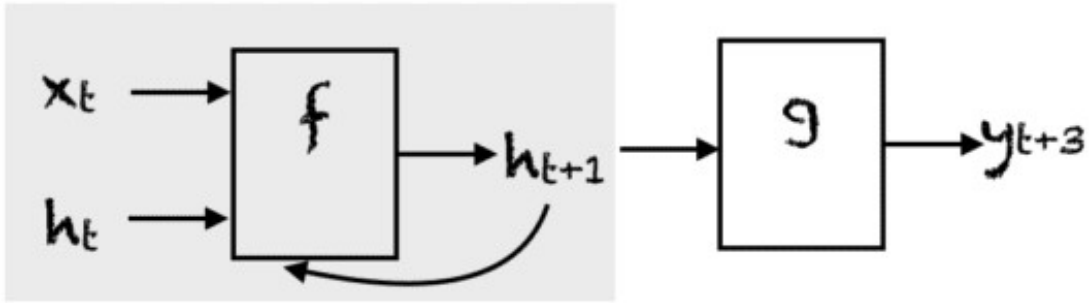
- **Many-to-many** models map several inputs to several outputs. Machine translation and name entity recognition are powered by many-to-many RNNs, where multiple words or sentences can be structured into multiple different outputs (like a new language or varied categorizations).



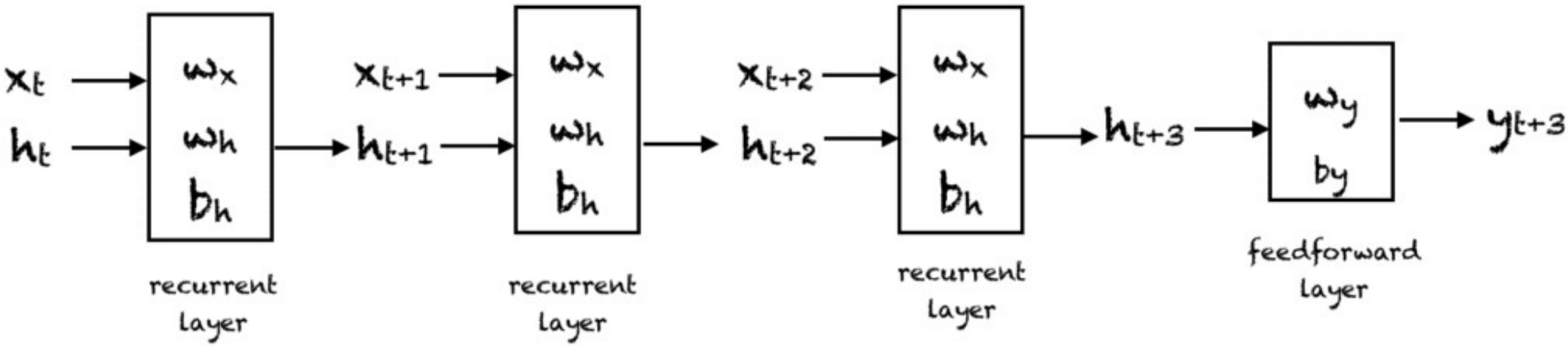
Unrolling Recurrent Neural Networks



Unrolling Recurrent Neural Networks



Unfolding the feedback loop in gray for k=3



Updating Hidden State in RNNs

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

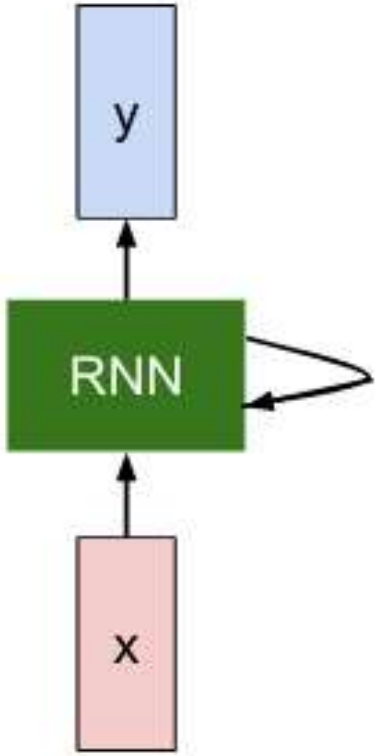
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

input vector at some time step



Generating Output in RNNs

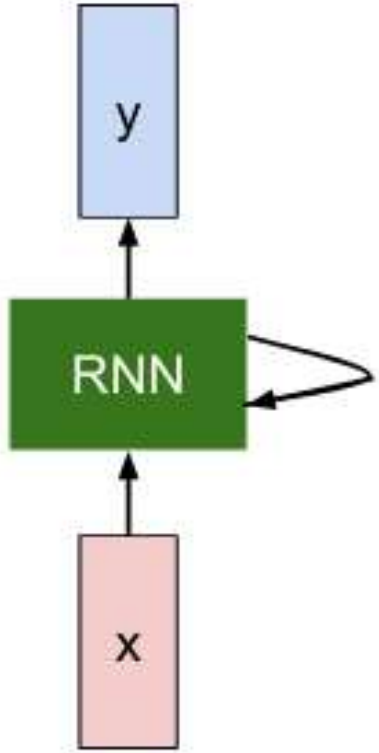
We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$y_t = f_{W_{hy}}(h_t)$$

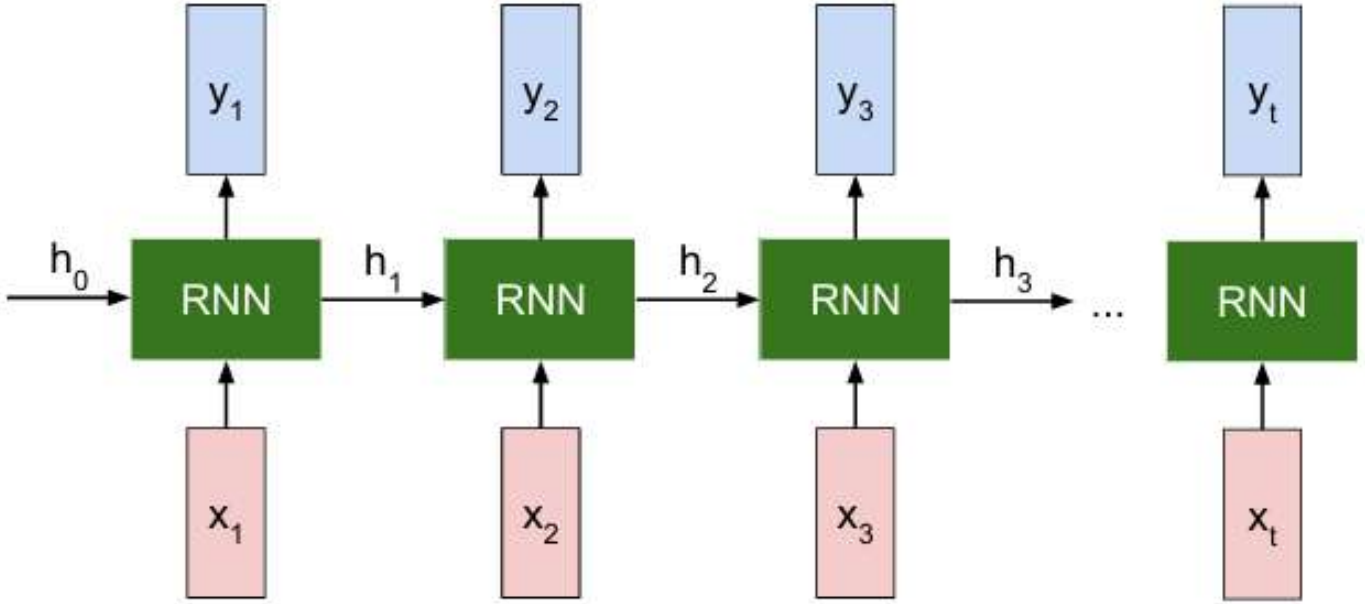
output

new state

another function with parameters W_o



Recurrent Neural Network



$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.

Recurrent Neural Networks

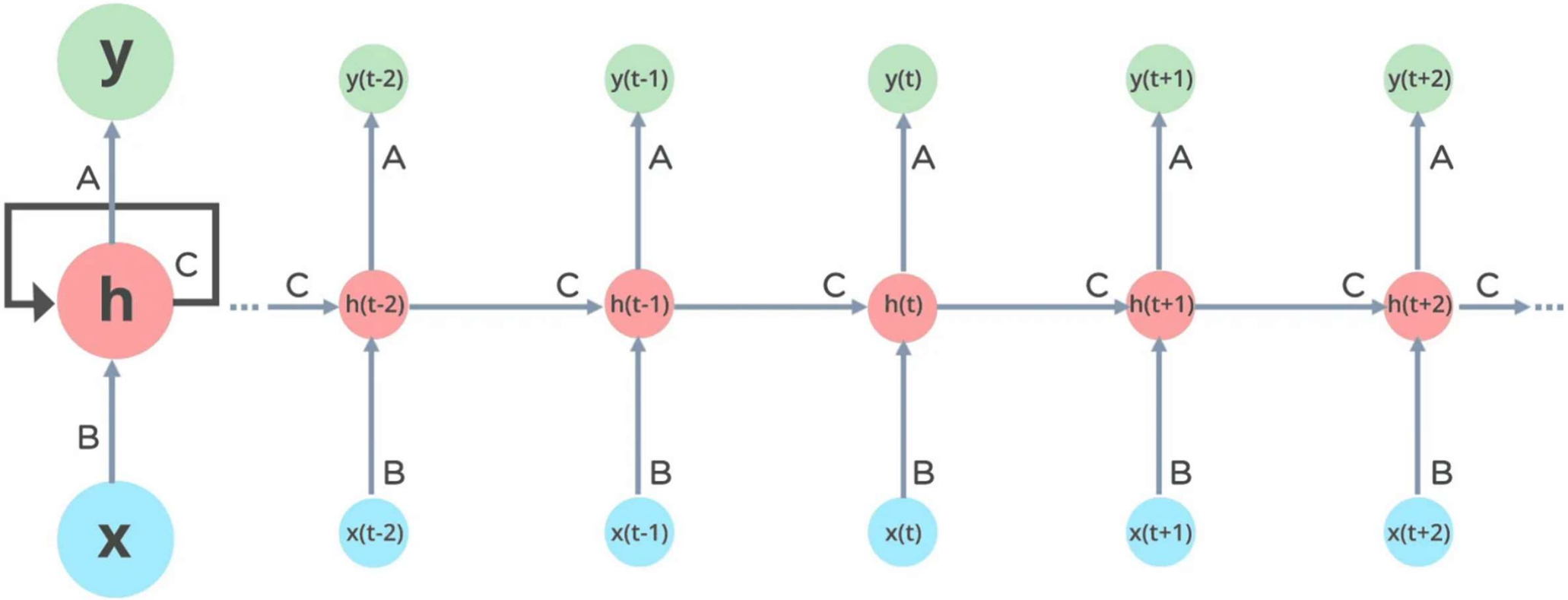
- A distinguishing characteristic of recurrent networks is that they **share parameters across each layer of the network**.

- While feedforward networks have different weights across each node, recurrent neural networks share the same weight parameter within each layer of the network.

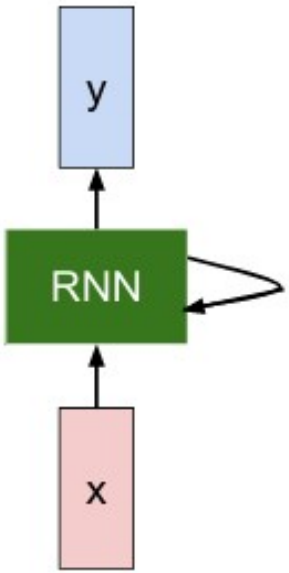
- That said, these weights are still adjusted through the processes of backpropagation and gradient descent to facilitate reinforcement learning.

Recurrent Neural Networks

- The Recurrent Neural Network will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters. Then, instead of creating multiple hidden layers, it will create one and loop over it as many times as required.



Recurrent Neural Networks



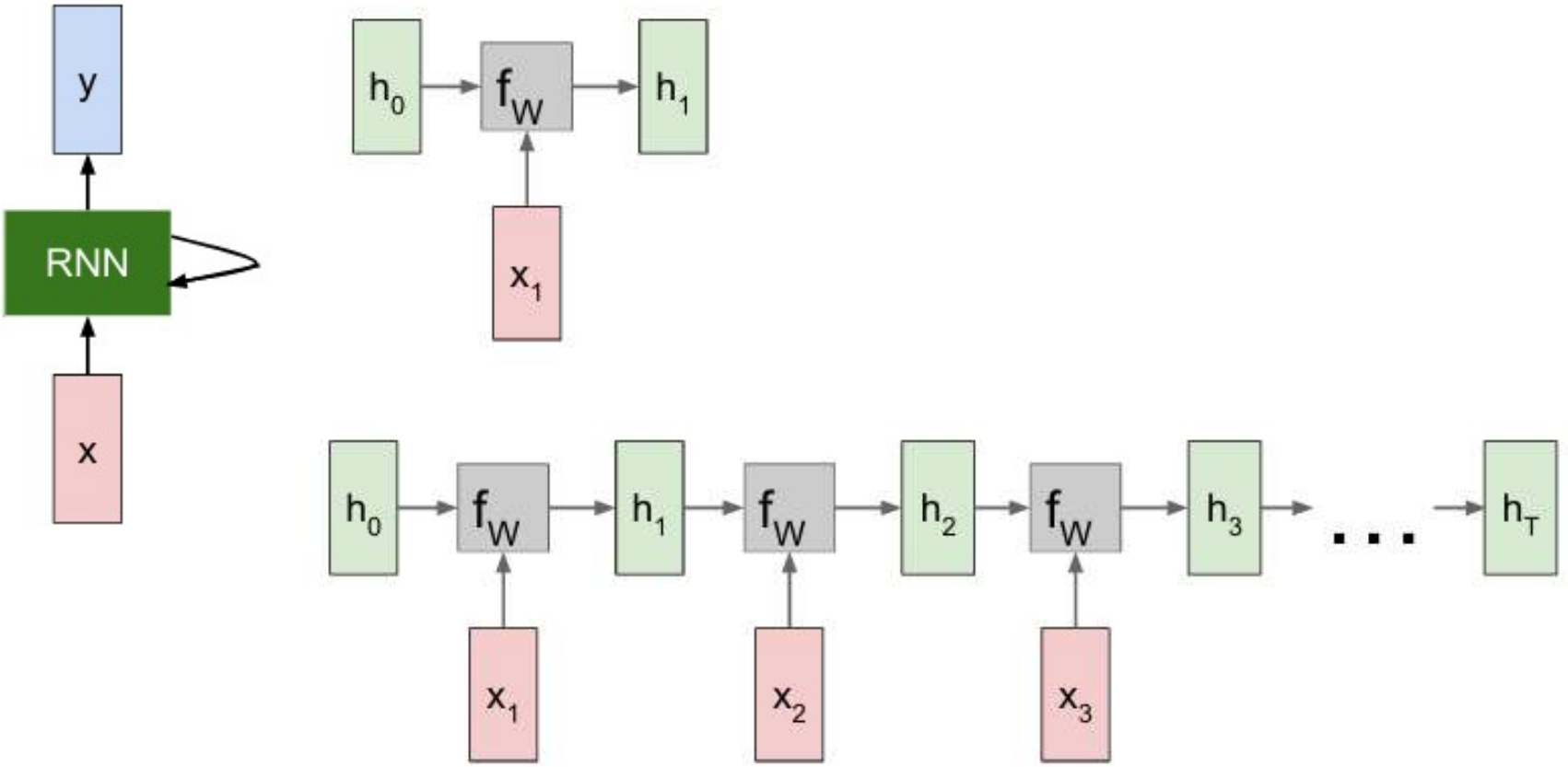
$$h_t = f_W(h_{t-1}, x_t)$$

$$\downarrow$$

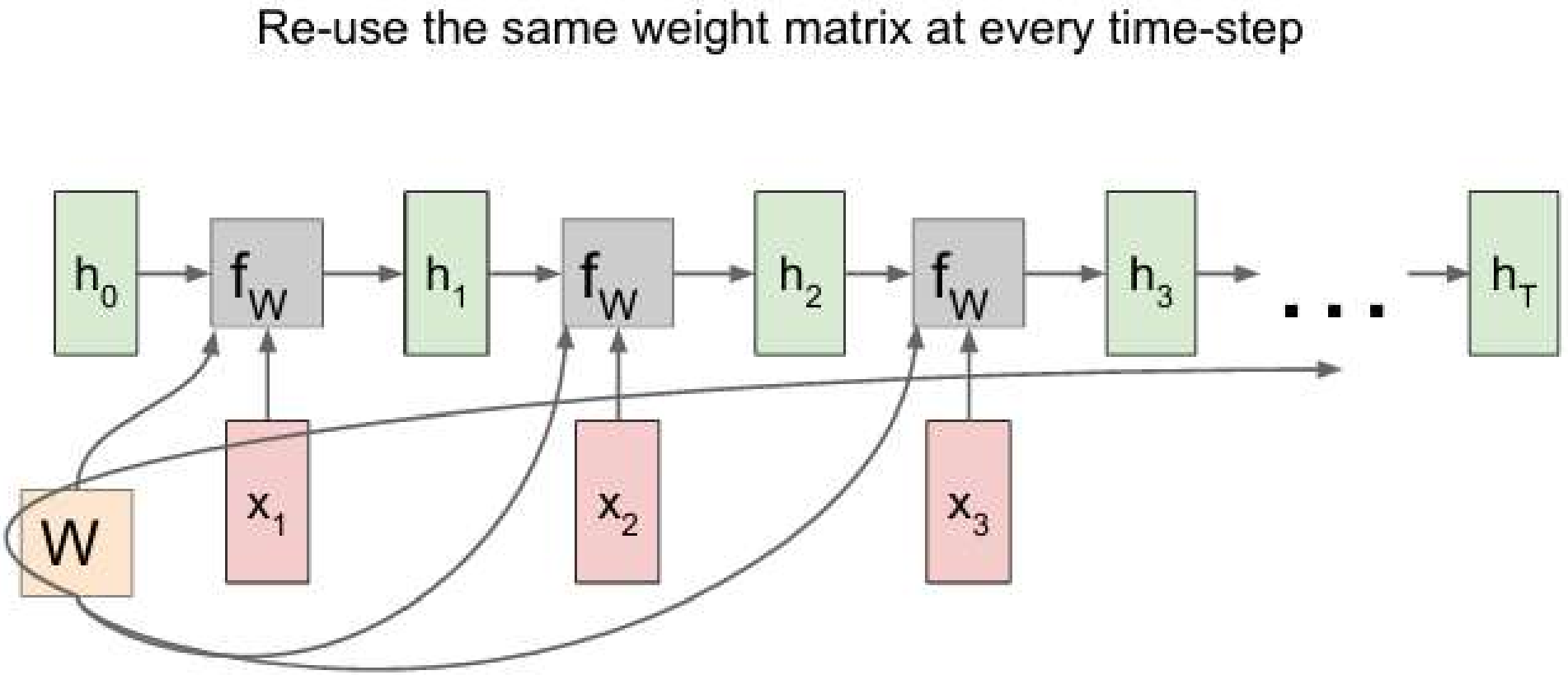
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

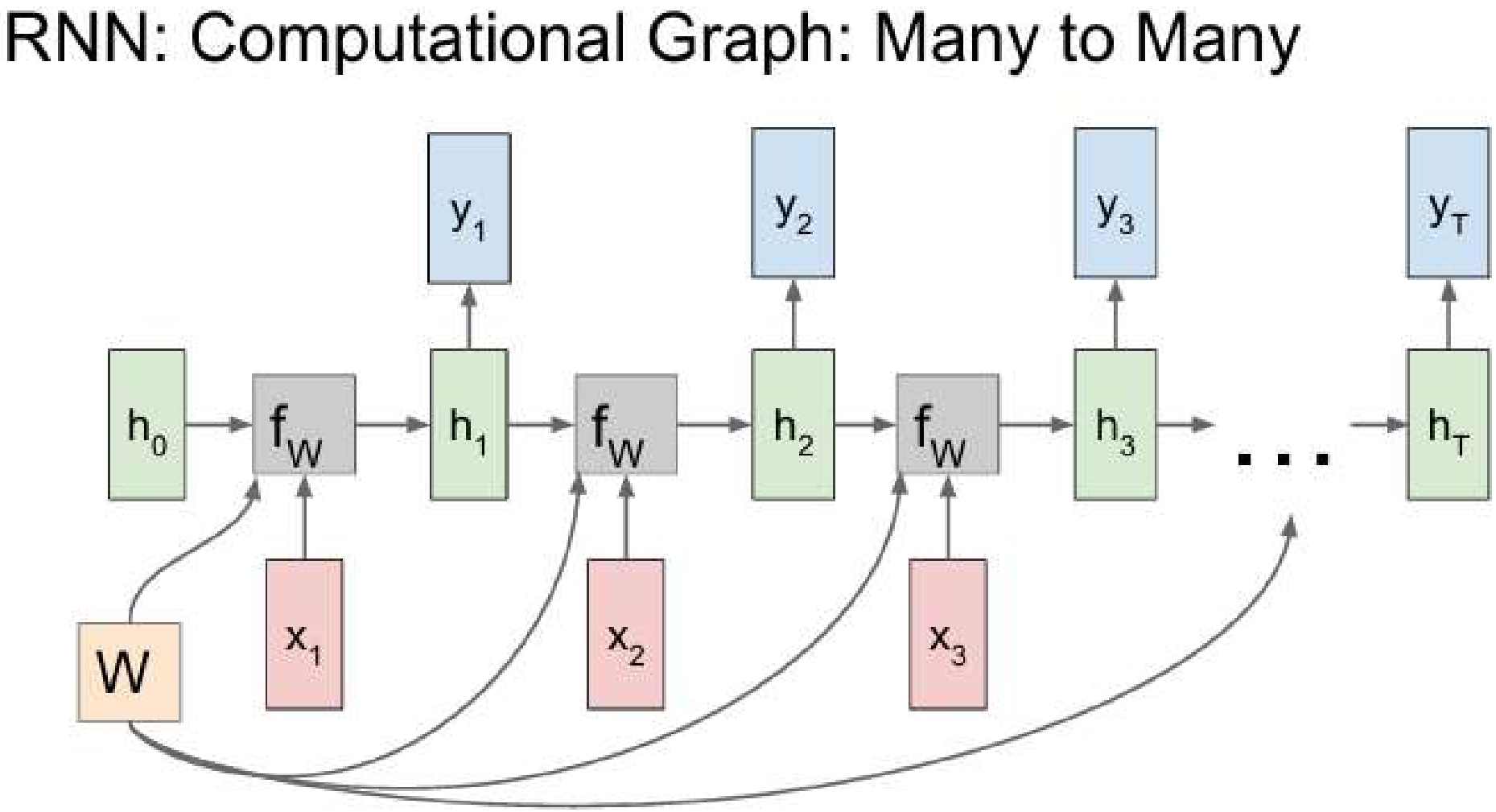
RNN: Computation Graph



RNN: Computation Graph



RNN: Computation Graph



RNN: Probabilistic Interpretation

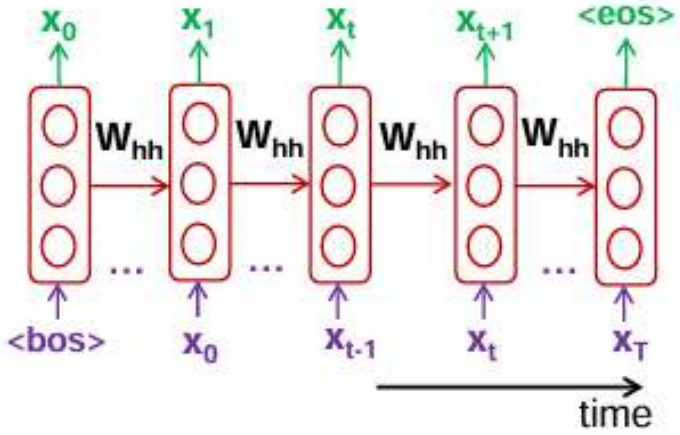
RNN as a generative model

- induces a set of procedures to model

the conditional distribution of $\mathbf{x}_{t+1}$ given $\mathbf{x}$<=t

for all t = 1, …,T

$$P(x) = P(x_1, \dots x_T) = \sum_{t=1}^{T} P(x_t | x_{t-1}, x_{t-2}, .. x_1)$$



Generative Recurrent Neural Network (RNN)

- Think of the output as the probability distribution of the $\mathbf{x}_t$ given the previous ones in the sequence
- Training: Computing probability of the sequence and Maximum likelihood training

$$L_t = -\log P(x_t | x_{t-1}, x_{t-2}, .. x_1)$$

Long Term and Short Dependencies

Short Term Dependencies

→ need recent information to perform the present task.

For example in a language model, predict the next word based on the previous ones.

*"the clouds are in the **?**"* → *'sky'*

*"the clouds are in the **sky**"*

→ Easier to predict 'sky' given the context, i.e., *short term dependency*

Long Term Dependencies

→ Consider longer word sequence "I grew up in France…..…… I speak fluent ***French***."

→ Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back.

Backpropagation Through Time (BPTT)

Backpropagation Through Time (BPTT)

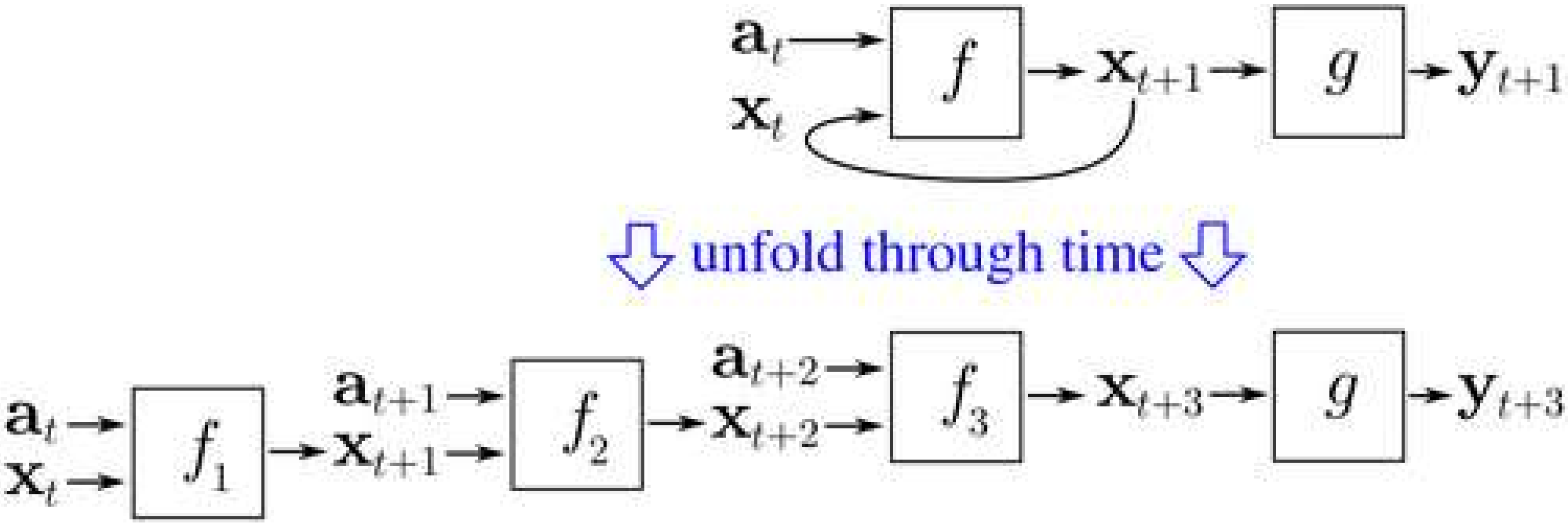
- Backpropagation through time (BPTT) is a gradient-based technique for training certain types of recurrent neural networks.

- The training data for a recurrent neural network is an ordered sequence of K input-output pairs,

$$\langle \mathbf{a}_0, \mathbf{y}_0 \rangle, \langle \mathbf{a}_1, \mathbf{y}_1 \rangle, \langle \mathbf{a}_2, \mathbf{y}_2 \rangle, \cdots, \langle \mathbf{a}_{k-1}, \mathbf{y}_{k-1} \rangle$$

- An initial value must be specified for the hidden state x_0 typically chosen to be a zero vector.

Backpropagation Through Time (BPTT)



BPTT begins by unfolding a recurrent neural network in time. The unfolded network contains K inputs and outputs, but every copy of the network shares the same parameters. Then, the backpropagation algorithm is used to find the gradient of the loss function with respect to all the network parameters.

Backpropagation Through Time (BPTT)

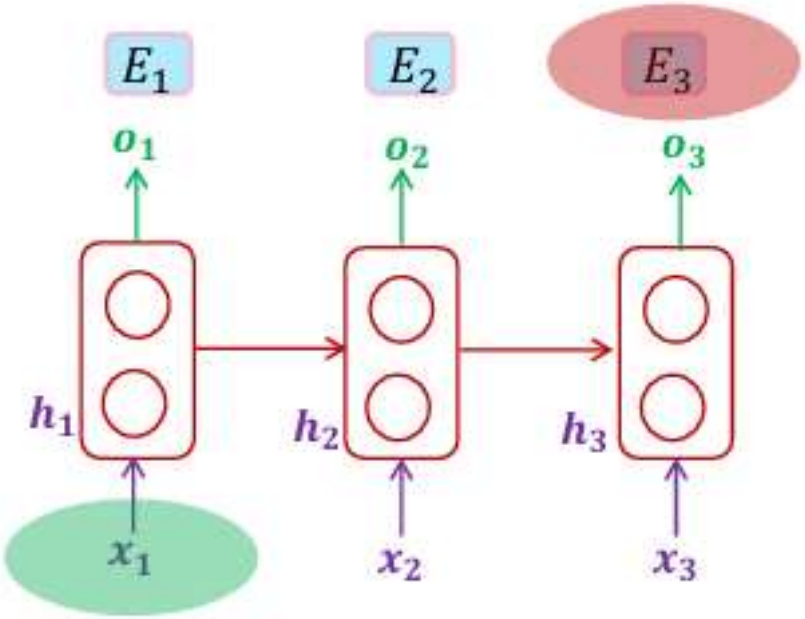
Training algorithm in time domain:

- The forward pass builds up a stack of the activities of all the units at each time step
- The backward pass peels activities off the stack to compute the error derivatives at each time step.
- After the backward pass we add together the derivatives at all the different times for each weight.

Forward through entire sequence → compute loss
Backward through entire sequence → compute gradient

Backpropagation Through Time (BPTT)

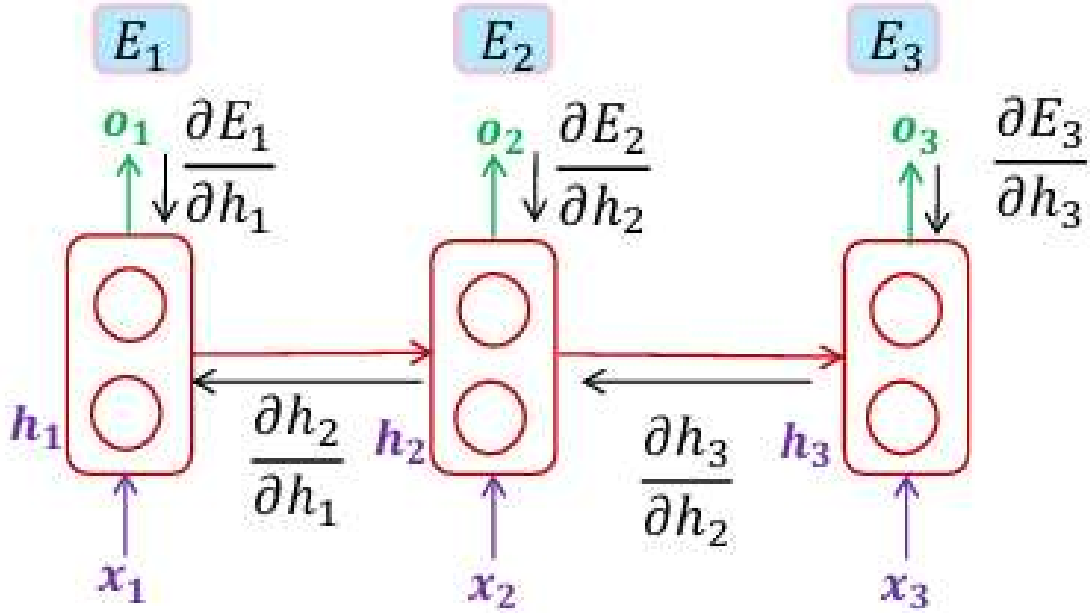
The **output** at time t=T is **dependent** on the inputs from **t=T to t=1**



Direction of Forward pass

Backpropagation Through Time (BPTT)

The **output** at time t=T is **dependent** on the inputs from **t=T to t=1**

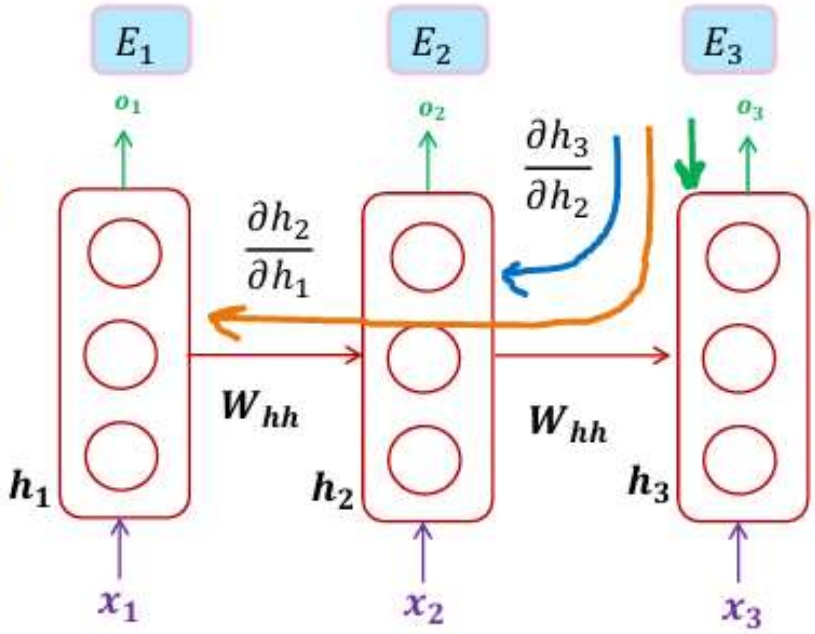


Direction of *Backward* pass (via partial derivatives)
--- gradient flow ---

Backpropagation Through Time (BPTT)

$$\frac{\partial E_3}{\partial W_{hh}} = \sum_{k=1}^{3} \frac{\partial E_3}{\partial h_3} \frac{\partial h_3}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

$$= \frac{\partial E_3}{\partial o_3} \frac{\partial o_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_{hh}} + \frac{\partial E_3}{\partial o_3} \frac{\partial o_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_{hh}} + \frac{\partial E_3}{\partial o_3} \frac{\partial o_3}{\partial h_3} \frac{\partial h_3}{\partial h_3} \frac{\partial h_3}{\partial W_{hh}}$$



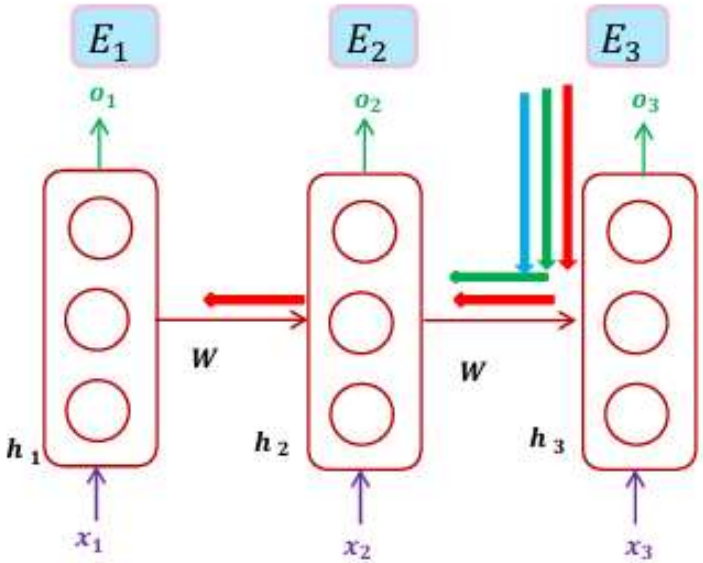
Backpropagation Through Time (BPTT)

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_3}\frac{\partial h_3}{\partial W}$$

$$= \lll 1 \quad + \quad \ll 1 \quad + \quad < 1$$

The gradients no longer depend on the past inputs…

since, the near past inputs dominate the gradient !!!



Total Gradient = **Gradient due to long term dependencies** + **Gradient due to short term dependencies**

Remark: *The gradients due to short term dependencies (just previous dependencies) dominates the gradients due to long-term dependencies.*

This means network will tend to focus on short term dependencies which is often not desired

Problem of Vanishing Gradient

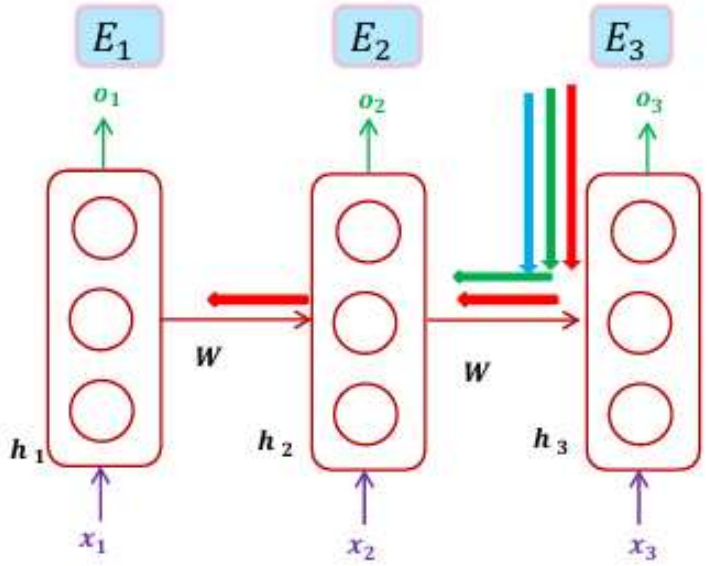
Backpropagation Through Time (BPTT)

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial E_3}{\partial h_3}\frac{\partial h_3}{\partial h_3}\frac{\partial h_3}{\partial W}$$

$$= \gg\gg 1 \quad + \quad \ggg 1 \quad + \quad \ggg 1$$

= Very large number, i.e., NaN

Problem of Exploding Gradient



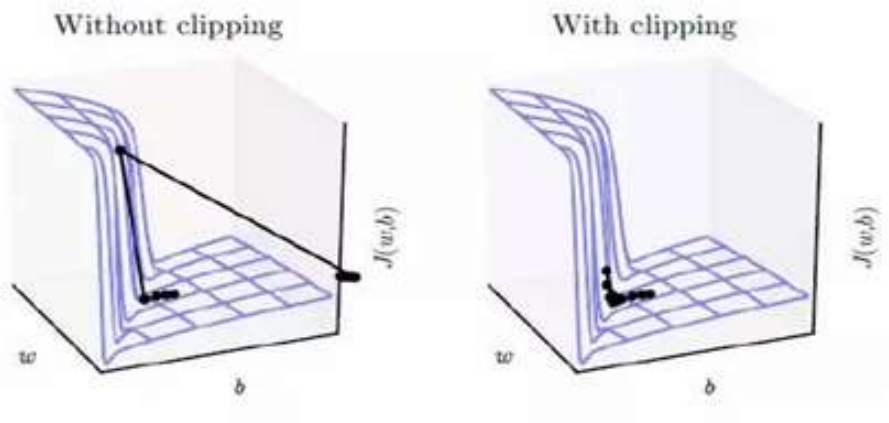
Dealing with Exploding Gradients: Gradient Clipping

Scaling down the gradients

- rescale norm of the gradients whenever it goes over a threshold

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if



- Proposed clipping is simple and computationally efficient,
- introduce an additional hyper-parameter, namely the threshold

Dealing with Vanishing Gradients

- As discussed, the gradient vanishes due to the recurrent part of the RNN equations.

$$h_t = W_{hh} \mathrm{h}_{t-1} + \text{some other terms}$$

- **What if Largest Eigen value of the parameter matrix becomes 1, but in this case, memory just grows.**
- **We need to be able to decide when to put information in the memory**

Applications of RNNs

- **Natural Language Processing (NLP):**
- **Language Modeling:** RNNs can predict the next word in a sentence, useful for tasks like text generation and autocomplete.
- **Machine Translation:** RNNs can be used for translation tasks, with one RNN encoding the input sentence and another decoding it in the target language.
- **Sentiment Analysis:** RNNs can analyze text sentiment by capturing contextual information from words in a sequence.

Applications of RNNs

- **Speech Recognition and Synthesis:**
- **Speech-to-Text:** RNNs are employed to convert spoken language into written text, making them the backbone of speech recognition systems.
- **Text-to-Speech:** RNNs can generate human-like speech from text input, improving voice assistants and accessibility tools.

Applications of RNNs

- **Time-Series Analysis and Forecasting:**
- **Financial Forecasting:** RNNs can predict stock prices, currency exchange rates, and other financial variables.

- **Weather Prediction:** RNNs can analyze historical weather data to forecast future weather conditions.

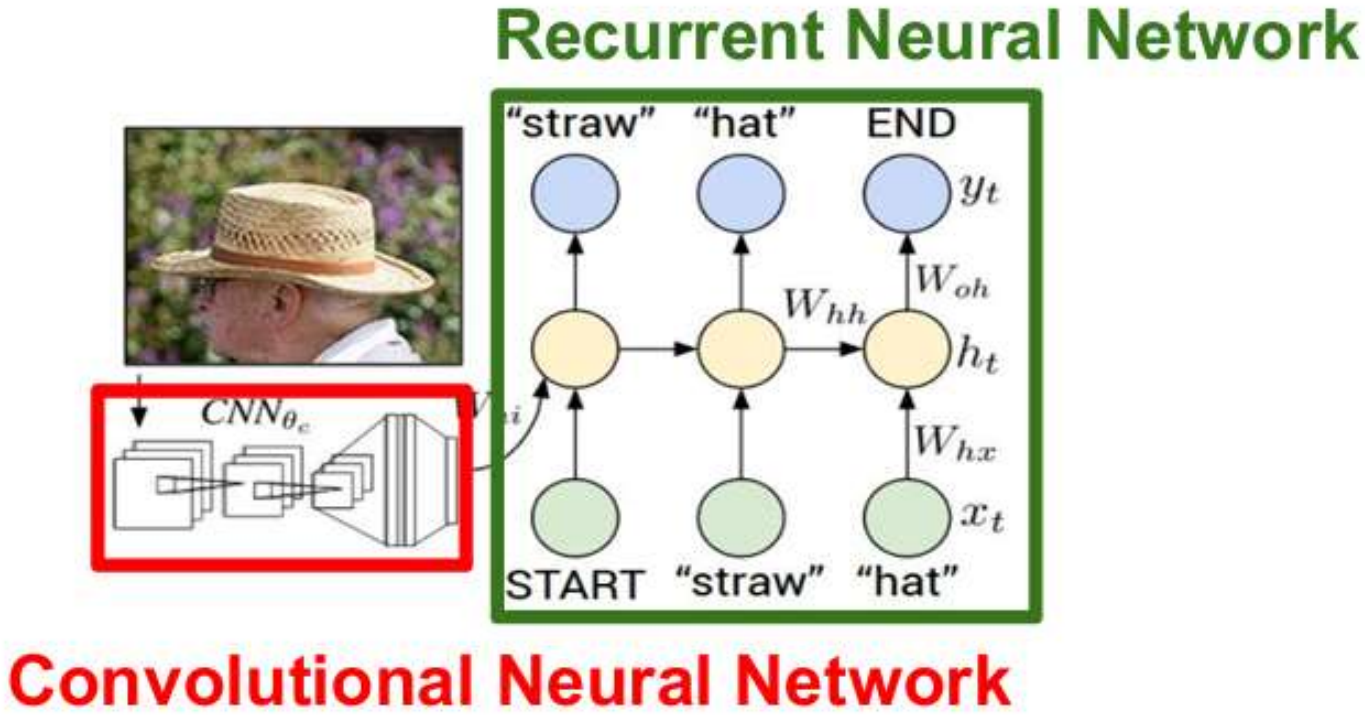
Applications of RNNs

- **Video Analysis and Action Recognition:**
- **Video Understanding:** RNNs can process frames in a video sequence to understand actions, objects, and activities.

- **Gesture Recognition:** RNNs can recognize hand gestures and movements in videos for applications like sign language translation.

Applications of RNNs

- **Captioning of images:**
- The process of creating text that describes the content of an image is known as image captioning. The image's content can depict the object as well as the action of the object on the image.



Advantages of RNNs

- **Sequence Data Handling:** RNNs are inherently suited for sequential data, making them ideal for applications where context and the order of data points are crucial, such as in language translation or stock prediction.

- **Temporal Dependency Modeling:** They excel at capturing temporal dynamics and dependencies within the data, a capability essential for understanding complex sequences over time, thus providing nuanced insights into data trends and patterns.

Advantages of RNNs

- **Flexibility in Sequence Length:** Unlike other neural networks, RNNs can handle inputs of varying lengths, offering flexibility when working with diverse datasets.

- **Parameters Sharing:** The same weights are used across timestamps, making them efficient in terms of memory and computational requirements.

Disadvantages of RNNs

- **Vanishing and exploding gradients:** Due to the way the layers are arranged in an RNN model, the sequence typically gets rather long. This results in the model training with exploding or null weights, leading to gradient concerns.

- **Slow and complex training:** In comparison with other networks, RNN takes a lot of time in training. To add to that, the training is quite complex and difficult to implement.

- **Memory Constraints:** Processing large sequences or large volumes of data can lead to memory bottlenecks. RNNs maintain states for each element in the input sequence, which can quickly consume available memory, impacting performance and scalability.

Long Short Term Memory

Long Term and Short Dependencies

Short Term Dependencies

→ need recent information to perform the present task.

For example in a language model, predict the next word based on the previous ones.

*"the clouds are in the **?**"* → *'sky'*

*"the clouds are in the **sky**"*

→ Easier to predict 'sky' given the context, i.e., *short term dependency*

Long Term Dependencies

→ Consider longer word sequence "I grew up in France…..…… I speak fluent ***French***."

→ Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back.

Dealing with Vanishing Gradients

- As discussed, the gradient vanishes due to the recurrent part of the RNN equations.

$$h_t = W_{hh} \mathrm{h}_{t-1} + \text{some other terms}$$

- **What if Largest Eigen value of the parameter matrix becomes 1, but in this case, memory just grows.**
- **We need to be able to decide when to put information in the memory**

Long Short Term Memory (LSTM)

- A standard RNN that has both “long-term memory” and “short-term memory”.

- LSTM networks combat the RNN's vanishing gradients or long-term dependence issue.

- The weights and biases of connections in the network change once per training episode, analogous to how physiological changes in synaptic strengths store long-term memories; activation patterns in the network change once per time step, analogous to how an instantaneous change in electrical firing patterns in the brain stores short-term memories.

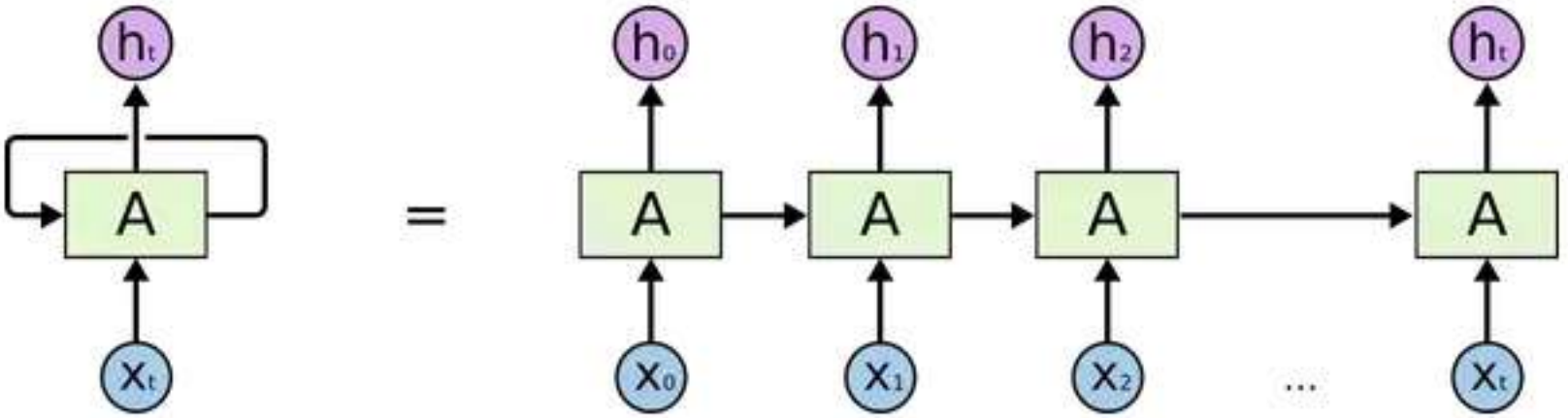
Long Short Term Memory (LSTM)

- For example, if an RNN is asked to predict the following word in this phrase, "have a pleasant _____," it will readily anticipate "day.“

- I am going to buy a table that is large in size, it’ll cost more, which means I have to _____ down my budget for the chair.

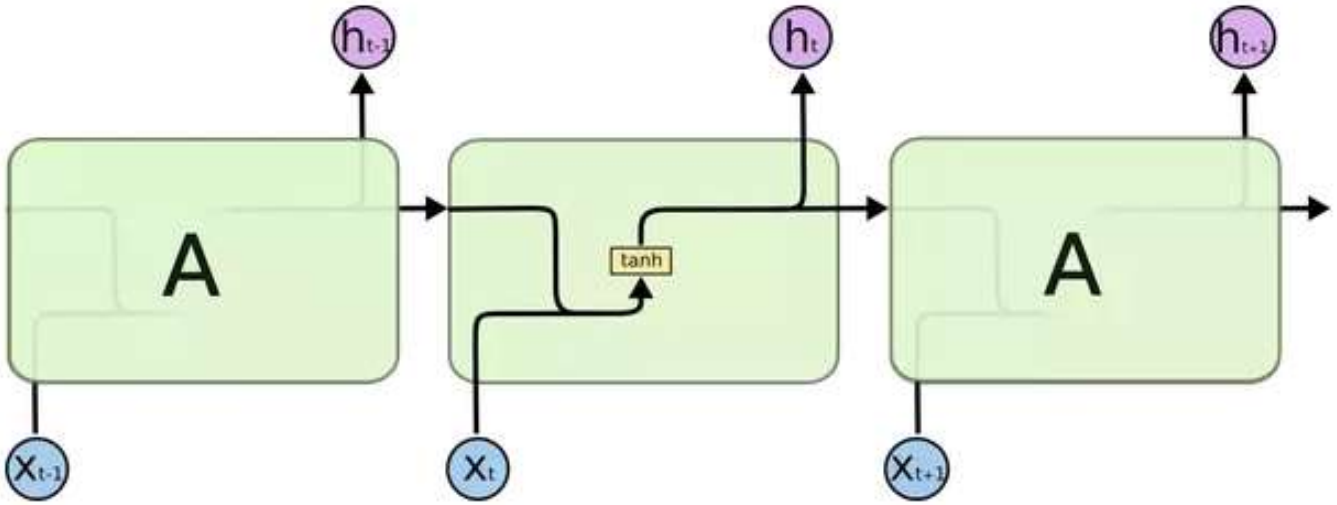
- If there is no valuable data from other inputs (previous words of the sentence), LSTM will **forget** that data and produce the result “Cut down the budget.”

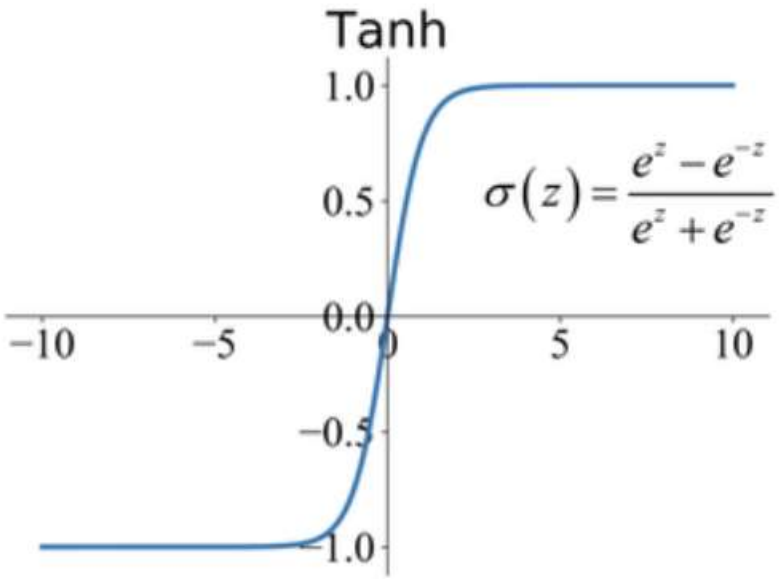
RNN vs. LSTM



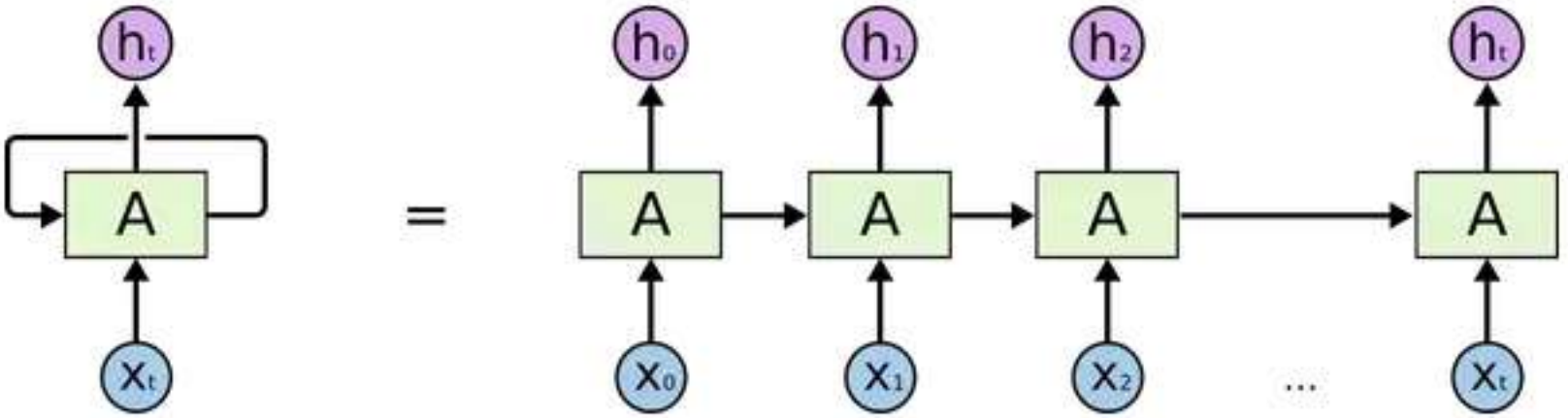
- Basically, we are feeding in a sequence of inputs. The hope is that the states of the "cell" contains information from all of the inputs that have been fed in up to that point, i.e., all of the Xs that have been fed in have a say in the state of A. Think of it like A is supposed to listen to, more or less, every one of the Xs.

RNN vs. LSTM


h_{t-1}
h_t
h_{t+1}
A
tanh
A
X_{t-1}
X_t
X_{t+1}

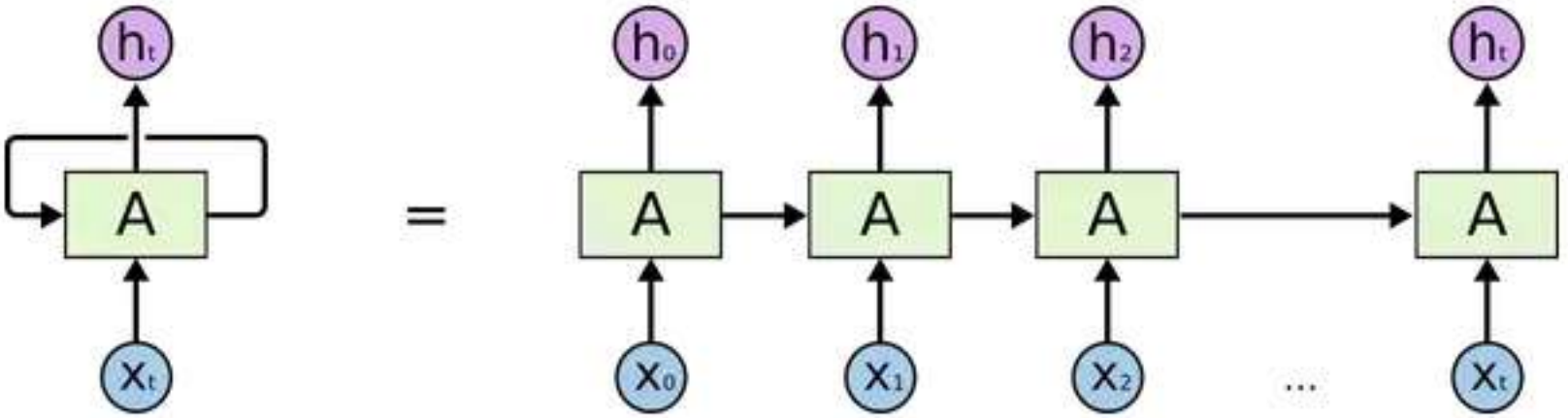

Tanh
1.0
0.5
0.0
−0.5
−1.0
−10
−5
0
5
10
$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

RNN vs. LSTM



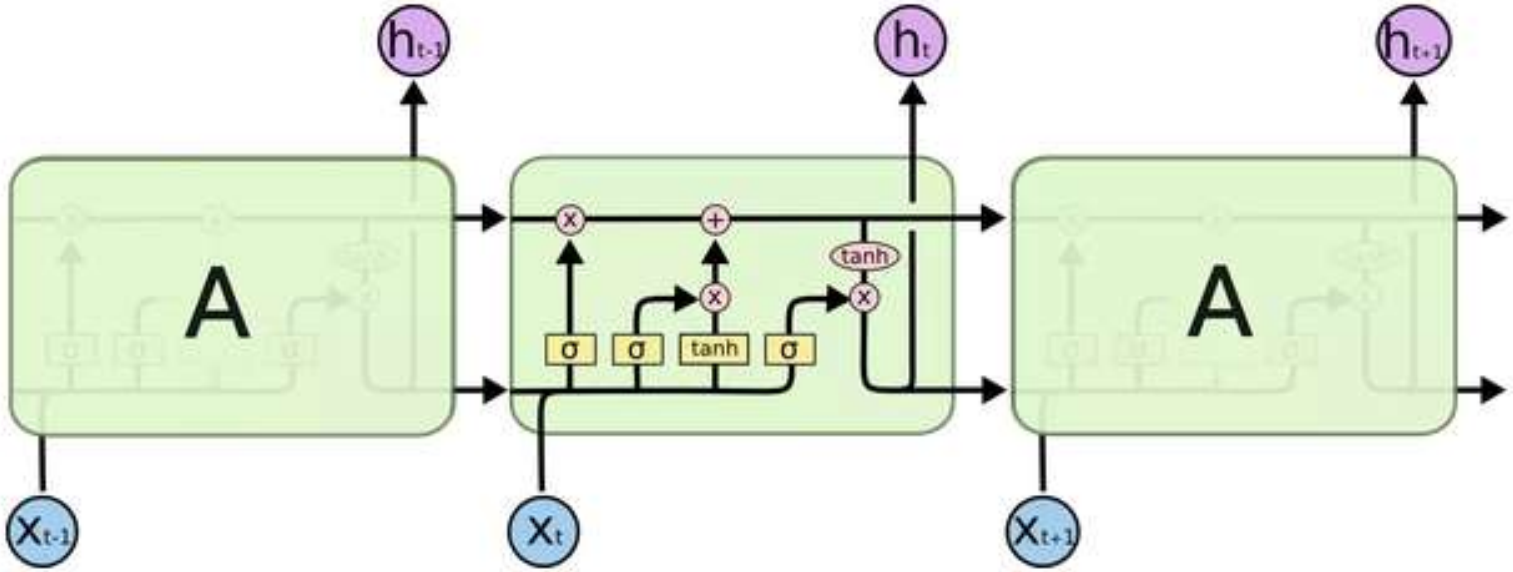
- X0: “Hey A! *Important info*”
- A: “Okay. Got it.”
- X1: “Hey A! *Irrelevant info*”
- A: “Okay. Got it.”
- X2: “Hey A! *Important info*”
- A: Okay. Got it.”

RNN vs. LSTM



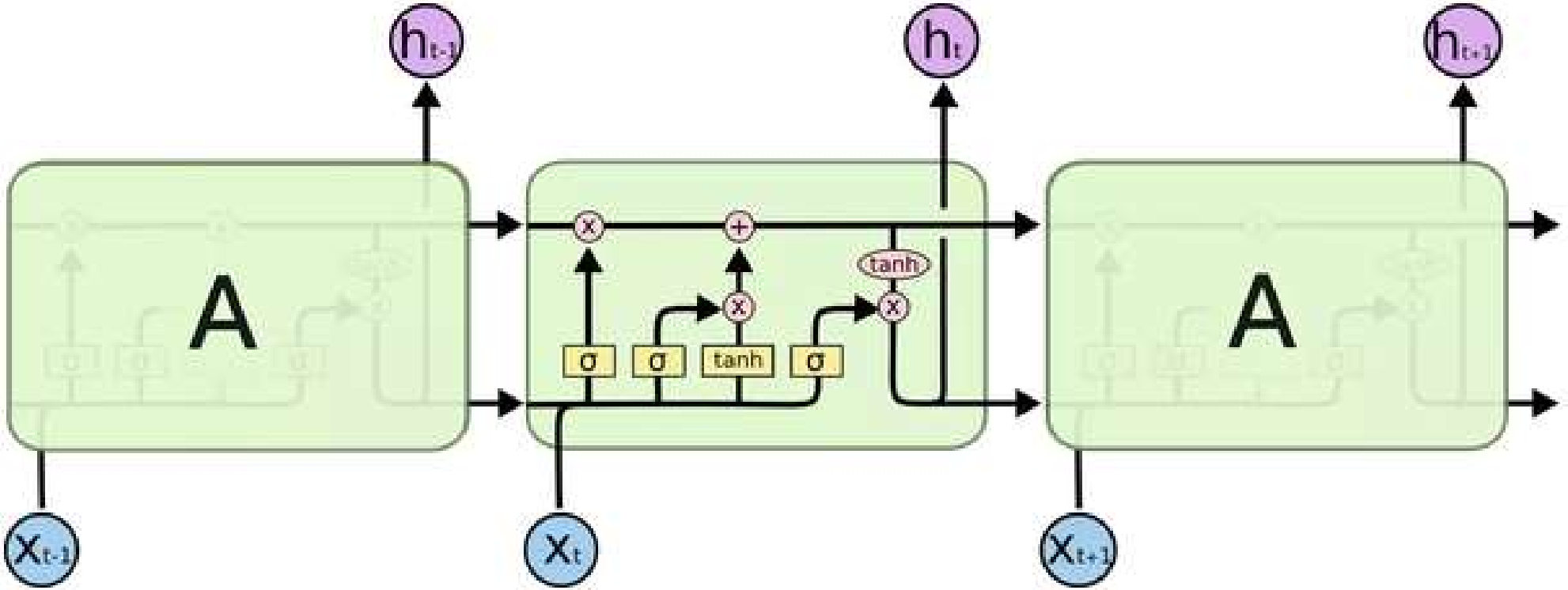
- So, in all likelihood, it mostly **just remembers** what the later Xs said, i.e., the things said by the Xs at the start of the sequence have little to no influence on what A remembers at the end.

RNN vs. LSTM

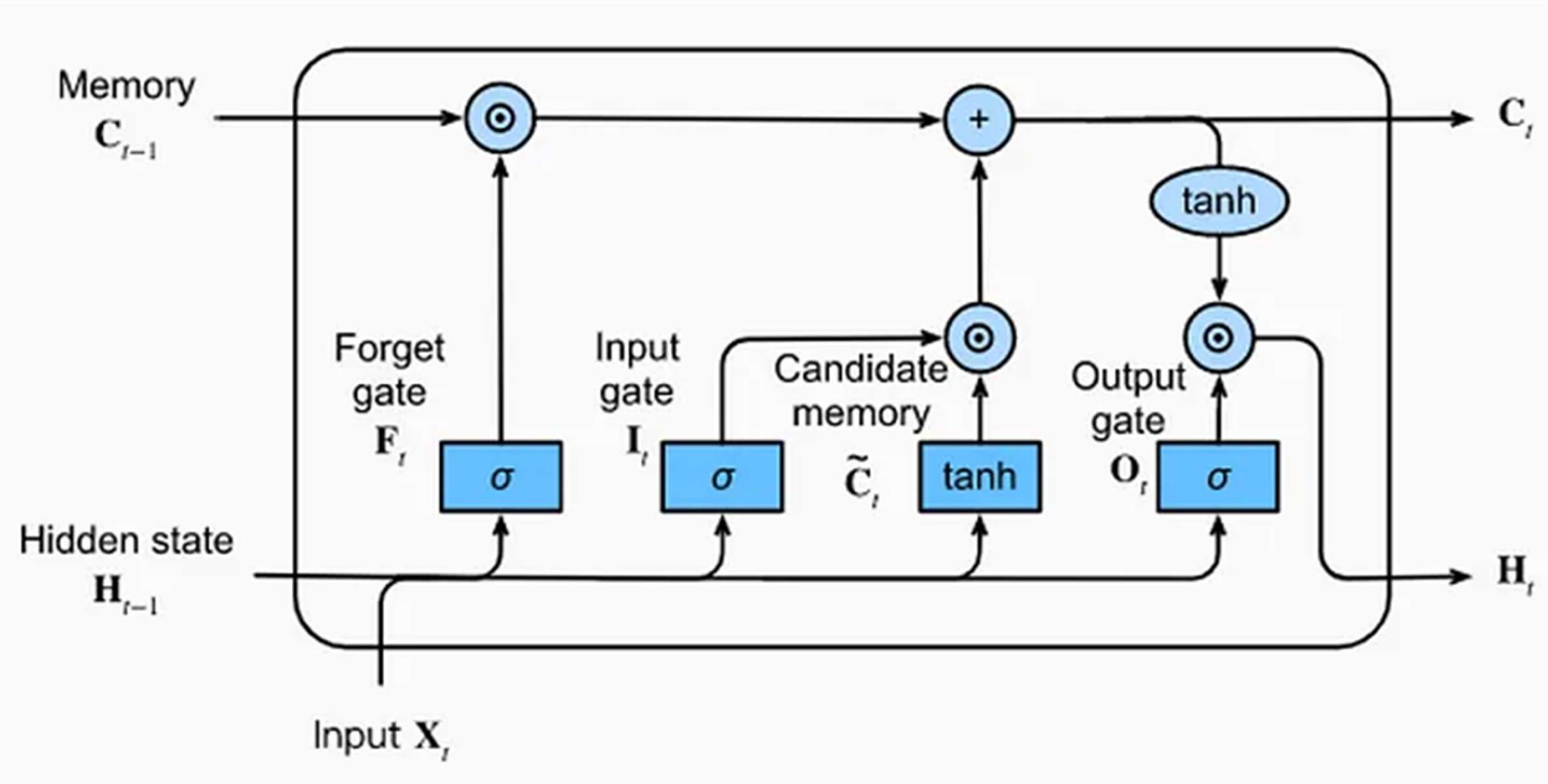


- X0: “Hey A! *Important info*”
- A: “Okay. Got it.”
- X1: “Hey A! *Irrelevant info*”
- A: “Okay. Forget it.”
- X2: “Hey A! *Important info*”
- A: Okay. Got it.”

LSTM Architecture



LSTM Architecture



Summary

- Recurrent Neural Networks